History of C Language

Introduction..... 1
Standardization of C.....2
What is Standardization?..... 2
Why is Standardization Needed?.....2
Different C Standards (First to Latest).....2
What is a Compiler?.....3
Functions of Compiler..... 3
Different C Compilers..... 3
Steps in Compilation Process..... 3
1. Preprocessing.....3
2. Compilation.....4
3. Assembling.....4
4. Linking..... 4
5. Loading.....5
Where Files Are Stored?..... 6
Windows vs Linux Compilation Files..... 7
Different Types of Environment.....8
Hosted Environment.....8
Freestanding Environment..... 10
What is an IDE?..... 12
Features:..... 12
Types of Computer Languages..... 12
1. Low-Level Language..... 12
2. Middle Level Language..... 13
3. High-Level Language..... 13
MCQs on C Language..... 14

Introduction

The **C language** was developed by Dennis Ritchie at AT&T Bell Laboratories in the early 1970s.

- Initially, the **UNIX operating system** was developed on the PDP-7 using **assembly language** and **PL/1**.
- While porting UNIX to the PDP-11, the **B language** (developed by Ken Thompson) was used.

- However, **B language had limitations** and was not efficient for system programming.
- Therefore, Dennis Ritchie developed the **C language**, improving upon B by adding:
    - Data types
    - Better memory handling
    - Structured programming features

As a result, **C became the primary language for UNIX development** and later a widely used general-purpose programming language.

Standardization of C

What is Standardization?

Standardization means defining **rules and specifications** for a language so that programs behave the same on all systems.

Why is Standardization Needed?

- To ensure **portability** (same program runs on different machines)
- To avoid **compiler differences**
- To maintain **consistency and reliability**
- To support **future development**

Different C Standards (First to Latest)

1. **K&R C (1978)**
    - By Dennis Ritchie & Brian Kernighan
    - First informal standard
2. **ANSI C (C89)**
    - Standardized by ANSI (American National Standards Institute) in 1989
3. **ISO C (C90)**
    - Adopted by ISO
4. **C99 (1999)**
    - Added new features like:
        - inline functions
        - new data types (long long)
5. **C11 (2011)**
    - Added:
        - Multithreading
        - Better safety features
6. **C17 (2018)**
    - Minor improvements and bug fixes
7. **C23 (Latest)**

- Modern updates and enhancements

What is a Compiler?

A **compiler** is a software tool that **converts C program (source code) into machine code**.

Example: GCC

Functions of Compiler

- Checks syntax errors
- Converts code into object code
- Optimizes program

Different C Compilers

- GCC
- Turbo C
- Clang
- Microsoft Visual C
- MinGW
- Intel C Compiler
- Tiny C Compiler

Steps in Compilation Process

1. Preprocessing

Done by **Preprocessor**

Work:

- Expands #define macros
- Includes header files (#include)
- Handles conditional compilation (#if, #ifdef)
- Removes comments

Input File:

- program.c

Output File:

- program.i (expanded source code)

2. Compilation

Done by compiler like GCC

Work:

- Checks syntax errors
- Converts code to **assembly language**
- Performs optimization

Input File:

- program.i

Output File:

- program.s (assembly code)

3. Assembling

Done by **Assembler**

Work:

- Converts assembly code into a machine code.
- Example:
  MOV AX, 10;
  MOV BX, 20;
  ADD AX, BX;

Input File:

- program.s

Output File:

- program.o (object file)

4. Linking

Done by **Linker**

Work:

- Combines object file with library files

- Resolves function calls like printf()
- Adds required library code

Input Files:

- program.o + libraries

Output File:

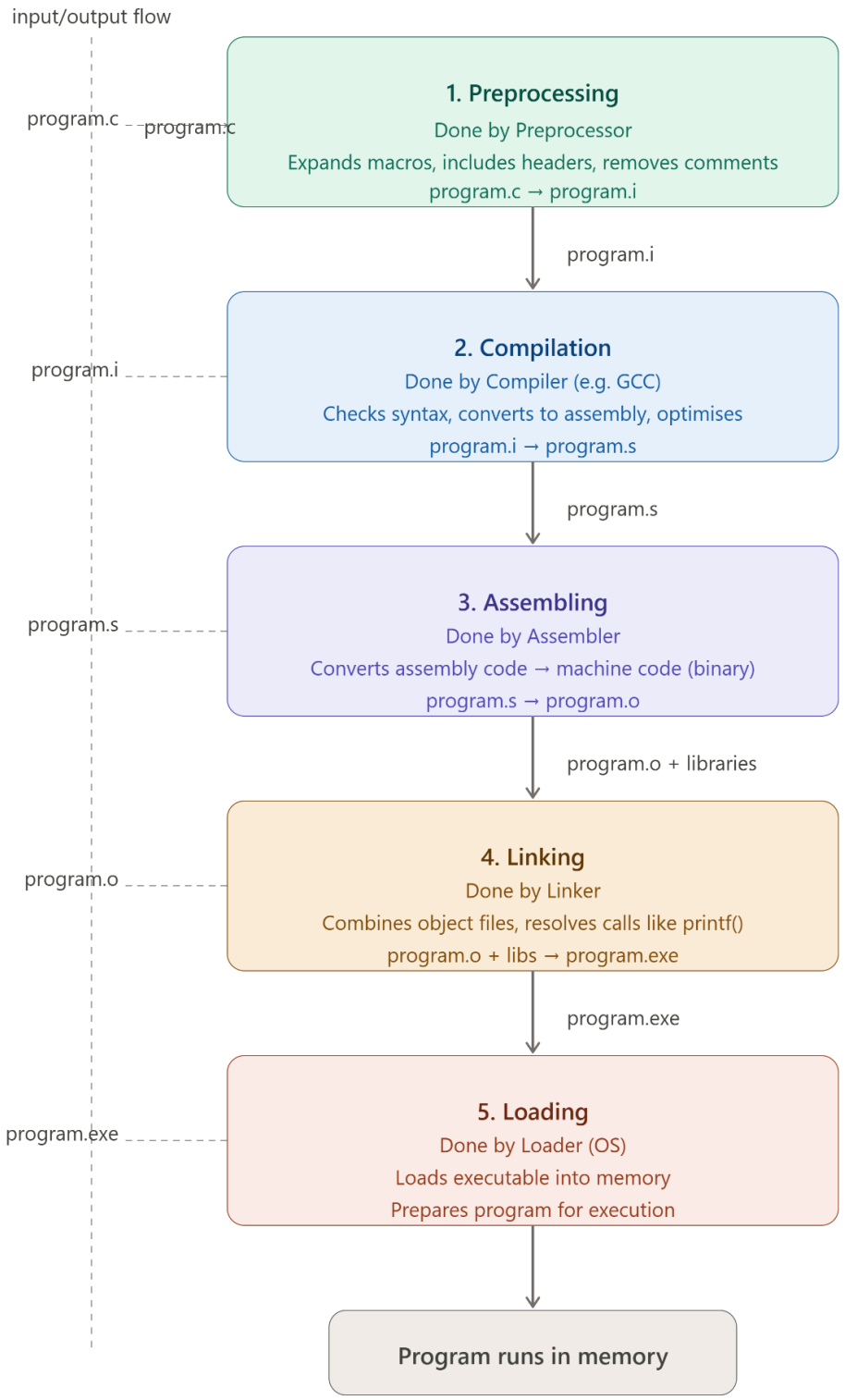
- program.exe (executable file)

5. Loading

Done by **Loader (OS)**

Work:

- Loads executable into memory
- Prepares program for execution


input/output flow
program.c
program.c
1. Preprocessing
Done by Preprocessor
Expands macros, includes headers, removes comments
program.c → program.i
program.i
program.i
2. Compilation
Done by Compiler (e.g. GCC)
Checks syntax, converts to assembly, optimises
program.i → program.s
program.s
program.s
3. Assembling
Done by Assembler
Converts assembly code → machine code (binary)
program.s → program.o
program.o + libraries
program.o
4. Linking
Done by Linker
Combines object files, resolves calls like printf()
program.o + libs → program.exe
program.exe
program.exe
5. Loading
Done by Loader (OS)
Loads executable into memory
Prepares program for execution
Program runs in memory

Where Files Are Stored?

Stage	File	Stored In
Source	program.c	Hard Disk / SSD
Preprocessing	program.i	Hard Disk / SSD
Compilation	program.s	Hard Disk / SSD
Assembling	program.o	Hard Disk / SSD
Linking	program.exe / a.out	Hard Disk / SSD
Loading	Process Image	RAM

Windows vs Linux Compilation Files

Stage	Windows	Linux
Source	.c	.c
Preprocessed	.i	.i
Assembly	.s	.s

Object	.obj	.o
Executable	.exe	a.out / executable
Debug File	.pdb	.debug / DWARF info

Different Types of Environment

There are two types of execution environments defined by the C standard:

1. **Hosted Environment**
2. **Freestanding Environment**

Hosted Environment

A **hosted environment** is a system where your C program runs **under the control of an operating system (OS)**.

Examples

Windows PC, Linux PC, macOS

Characteristics

- Has operating system.
- Has file system.
- Has standard library.
- Program execution begins at main().

Entry Point

c

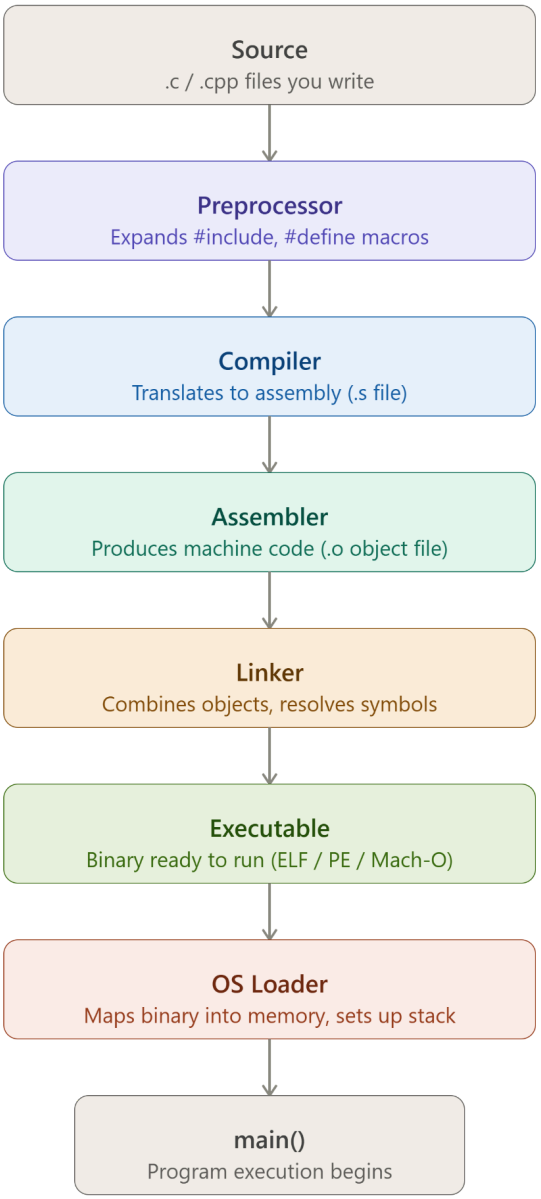
```c
int main(void)
{
}
```

Example Functions Available

c
printf()
scanf()

fopen()
malloc()

Compilation Flow



Freestanding Environment

A **freestanding environment** is a C programming environment that **does not require an operating system** and **does not have to provide the complete Standard C Library**. Programs run directly on the hardware, making this environment suitable for embedded systems such as microcontrollers.

Examples

PIC18F4580, LPC2148, STM32, ARM Cortex-M Microcontrollers

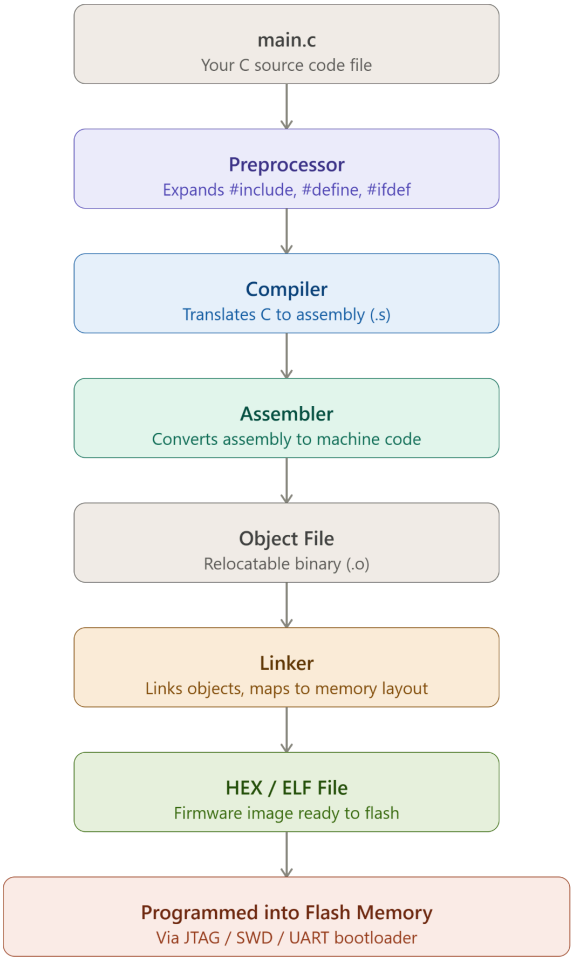
Program Entry

Execution usually starts from the Reset Vector, not directly from main(). A startup file executes first.

Startup File Tasks

1. Initialize stack pointer.
2. Initialize RAM variables.
3. Copy initialized data from Flash to RAM.
4. Clear BSS section.
5. Call main();

Typical Embedded Compilation Flow



Output Files

GCC / Linux:

program.elf
program.hex
program.bin

MPLAB XC8 / PIC18F4580:

project.hex
project.cof
project.elf

What is an IDE?

An **IDE (Integrated Development Environment)** is a software that provides **all tools required for programming in one place**.

Examples:

- Code::Blocks
- Eclipse
- Visual Studio Code
- DevC++

Features:

- Code editor
- Compiler integration
- Debugger
- Error highlighting

Types of Computer Languages

1. Low-Level Language

A low-level language is a programming language that provides direct control over hardware and system resources, with minimal abstraction from the computer's architecture.

Types:

- Machine Language (0s and 1s)
- Assembly Language

Features:

- Fast execution
- Difficult to understand
- Hardware dependent

Advantages:

- High execution speed.
- Efficient memory usage.
- Greater control over hardware.

2.Middle Level Language

A **middle-level language** is a programming language that combines the features of both **high-level** and **low-level** languages. It provides easy-to-understand syntax while also allowing direct access to hardware and memory.

Features:

- Easy to write and understand.
- Supports hardware-level programming.
- Provides good execution speed.
- Suitable for both system and application development.

Advantages:

- Efficient and fast execution.
- Better control over system resources.
- Portable across different platforms.

Example: C language is the most common middle-level language.

3. High-Level Language

A high-level language is a programming language that provides an easy-to-understand syntax and abstracts hardware details, allowing programmers to develop applications efficiently.

Examples:

- C++
- Java
- Python

Features:

- Easy to write and debug
- Portable
- Requires compiler/interpreter

Advantages:

- Faster program development.
- Easier maintenance and debugging.
- Improved readability.

MCQs on C Language

1. Who developed C language?
   A) James Gosling
   B) Dennis Ritchie
   C) Bjarne Stroustrup
   D) Ken Thompson

Answer: B

2. C language was developed at:
   A) Microsoft
   B) Google
   C) AT&T Bell Laboratories
   D) IBM

Answer: C

3. C language was mainly developed for:
   A) Game development
   B) Database
   C) UNIX operating system
   D) Web development

Answer: C

4. Which language was used before C?
   A) Java
   B) Python
   C) B language
   D) C++

Answer: C

5. ANSI C is also known as:
   A) C99
   B) C11
   C) C89
   D) C17

Answer: C

6. ANSI stands for:
   A) American National System Institute
   B) American National Standards Institute
   C) Advanced National Standard Institute
   D) None

Answer: B

7. ISO stands for:
   A) International System Organization
   B) Indian Standards Organization
   C) Internal Standard Office
   D) International Organization for Standardization

Answer: D

8. Which is the latest C standard?
   A) C99
   B) C11
   C) C17
   D) C23

Answer: D

9. A compiler converts:
   A) Machine code to C
   B) C to assembly
   C) C to machine code
   D) Binary to text

Answer: C

10. Which is a C compiler?
    A) Python
    B) Java
    C) GCC
    D) HTML

Answer: C

11. Compiler converts program:
    A) Line by line
    B) Whole at once
    C) Only functions
    D) Only variables

Answer: B

12. IDE stands for:
    A) Integrated Data Environment
    B) Internal Development Engine
    C) Integrated Development Environment
    D) Interface Development Editor

Answer: C

13. Which is an IDE?
    A) Code::Blocks
    B) Keyboard
    C) Mouse
    D) Printer

Answer: A

14. Which is NOT a C compiler?
    A) Turbo C
    B) Clang
    C) GCC
    D) MS Word

Answer: D

15. MinGW is:
    A) IDE
    B) Compiler
    C) OS
    D) Database

Answer: B

16. Machine language consists of:
    A) English words
    B) Symbols
    C) 0 and 1
    D) Alphabets

Answer: C

17. Which is a middle-level language?
    A) Assembly
    B) Machine
    C) C
    D) Binary

Answer: C

18. Low-level languages are:
    A) Easy to understand
    B) Hardware dependent
    C) Portable
    D) Slow

Answer: B